

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO3: Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Assessment Tool Weightage: 40%

Code of Conduct:

I **Kongara Jogesh (192525035)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kongara Jogesh

Case study

QUESTIONS: 4 Total Marks: 40

Name: Kongara Jogesh Reg No: 192525035

Case Study 1: Video Conferencing Quality Degradation (10 Marks)

A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an 8% packet loss, 180 ms jitter, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

1. Is the Problem Caused by the Transport Protocol or the Application Layer?

The symptoms described — voice distortion, frozen frames, and delayed communication under 8% packet loss and 180 ms jitter — point primarily to a transport-layer issue, specifically the behaviour of the protocol handling real-time media, compounded by how the application buffers incoming data.

- If the conferencing platform uses TCP (or a TCP-based media relay) for audio/video streams, every lost packet triggers retransmission and the receiver must wait for the missing segment before delivering data in order. With 8% loss, this causes exactly the kind of stalling and "frozen frame" behaviour described, because TCP prioritizes completeness over timeliness.
- High jitter (180 ms) indicates that packets are arriving at highly irregular intervals. This is a transport/network-level symptom (variable queuing delay), but it directly stresses the application's playout buffer — if the buffer is too small for this jitter level, frames arrive too late to be played smoothly, producing voice distortion and dropped video frames.
- Fluctuating bandwidth utilization suggests congestion-related loss, which affects transport-layer congestion control (if TCP is used) or is simply passed straight through to the application (if UDP is used) without any correction.

2. Justification Based on Real-Time Multimedia Communication Requirements

- Real-time multimedia applications prioritize timeliness over completeness: a late packet is effectively useless, since a stale audio/video frame cannot be meaningfully played back after its scheduled time.
- TCP's guarantees (in-order, lossless delivery via retransmission) work against real-time media: retransmitting a lost packet takes at least one round-trip time, which is often longer than the acceptable playout deadline, causing the visible freezing and delay.

- UDP, by contrast, does not retransmit or reorder, so occasional loss simply results in a brief glitch rather than a stall — but this shifts the responsibility for smoothness onto the application layer (jitter buffering, forward error correction, concealment).
- Given the symptoms (freezing rather than just minor glitches, and noticeable delay), it strongly suggests either TCP is being used for the media stream, or the application's jitter buffer is not correctly tuned to absorb 180 ms of jitter, or both factors are compounding each other.

3. Recommended Protocol-Level and Application-Level Improvements

- Protocol-level: Use UDP-based real-time transport (e.g., RTP/RTCP over UDP) instead of TCP for audio/video streams, since UDP avoids head-of-line blocking and lets the application decide how to handle loss.
- Protocol-level: Enable adaptive bitrate / congestion control mechanisms designed for real-time media (e.g., Google Congestion Control used in WebRTC) so bandwidth fluctuations are detected and the encoder adjusts its output rate before packet loss becomes severe.
- Application-level: Implement an adaptive jitter buffer that dynamically resizes based on measured jitter, so a 180 ms jitter spike is absorbed without producing distortion, while keeping added delay to a minimum during stable periods.
- Application-level: Use Forward Error Correction (FEC) and Packet Loss Concealment (PLC) so that a portion of the 8% lost packets can be reconstructed or masked without needing retransmission.
- Application-level: Prioritize audio over video when bandwidth is constrained (since audio distortion is far more disruptive to a meeting than a slightly degraded video frame), and use codecs that gracefully degrade quality rather than freezing.
- Network-level support: Apply Quality of Service (QoS) marking/prioritization for real-time media traffic on the corporate network to reduce queuing delay and jitter during peak business hours.

Case Study 2: High DNS Resolution Time for Global Web Services (10 Marks)

A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the average DNS lookup time is approximately 800 ms, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

1. Reasons Behind the High DNS Resolution Time

- Geographic distance to authoritative servers: If the company relies on a small number of centrally located DNS servers, employees in distant continents must send queries across long-haul links, adding significant round-trip latency.
- Lack of caching / low TTL values: If DNS records have very short Time-To-Live (TTL) values, resolvers must repeatedly query upstream/authoritative servers instead of serving cached answers, multiplying the lookup delay for every request.
- Recursive resolution overhead: Each uncached query may need to walk the full DNS hierarchy (root → TLD → authoritative server), and each hop across the globe adds additional round-trip time, compounding to the observed ~800 ms.
- Single point of resolution: If all queries funnel through one primary resolver/server rather than being served from a nearby location, every request incurs the full network delay, regardless of the requester's location.
- No use of Anycast or geo-distributed name servers: Without Anycast, a query from any region is routed to the same fixed server location instead of the nearest available replica.

2. Proposed DNS Architecture to Reduce Lookup Time Below 50 ms

- Anycast DNS: Deploy the same DNS server IP address at multiple geographically distributed points of presence (PoPs). Internet routing (BGP) automatically directs each user's query to the topologically nearest server, drastically cutting round-trip latency.
- TTL optimization: Set TTL values to a balanced duration (e.g., a few minutes to a few hours depending on how often records change) so that resolvers and client caches can reuse recent answers instead of re-querying for every request, while still allowing reasonably fast propagation of changes.
- DNS pre-fetching: Configure client browsers/applications to pre-resolve domain names for resources likely to be needed next (e.g., via DNS prefetching hints in the webpage), so the lookup happens in the background before it is actually needed, hiding the latency from the user.
- Distributed authoritative name servers: Deploy authoritative DNS servers in each major region (e.g., North America, Europe, Asia-Pacific) so that even uncached queries are resolved by a nearby authoritative source rather than crossing continents.
- Local caching resolvers: Deploy caching DNS resolvers close to each office/data center so that repeated lookups for the same domains are served almost instantly from cache.

3. Advantages and Limitations of the Proposed Architecture

Advantages

- Performance: Anycast routing combined with nearby authoritative/caching servers can realistically bring lookup times from ~800 ms down to well under 50 ms for most users, since queries are answered locally rather than traveling across continents.
- Scalability: Anycast inherently distributes query load across many server instances, allowing the system to handle growing global traffic without overloading a single location.
- Availability: With multiple Anycast nodes and distributed authoritative servers, the failure of one server/location is automatically routed around by BGP to the next-nearest healthy node, providing strong fault tolerance during server failures.

Limitations

- Operational complexity: Managing multiple synchronized Anycast nodes and distributed authoritative servers across regions requires more sophisticated monitoring, deployment, and consistency management than a single centralized server.
- TTL trade-off: Lower TTLs improve responsiveness to record changes and failover but increase query volume on authoritative servers; higher TTLs reduce load but slow down propagation of updates (e.g., during a failover), so the TTL value must be carefully tuned.
- Anycast routing unpredictability: Because Anycast relies on BGP routing decisions made by ISPs, the "nearest" server chosen is not always the geographically closest one, which can occasionally lead to suboptimal routing.
- Cost: Deploying and maintaining DNS infrastructure across multiple global points of presence increases infrastructure and operational cost compared to a single centralized deployment.

Case Study 3: Centralized DNS Bottleneck for Global Cloud Services (10 Marks)

A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly 800 ms because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic efficiently.

1. Impact of Centralized DNS Architecture on Application Performance

- Single point of failure: If the one primary DNS server becomes overloaded or fails (as is likely during high-traffic promotional events), all users worldwide are unable to resolve the service's domain name, effectively taking the entire application offline regardless of the health of the backend servers.

- High latency for distant users: Every user, no matter their location, must send their query to the same server location, so users far from that server experience long round-trip delays — explaining the observed ~800 ms lookup time.
- Poor scalability under load: A centralized server has a fixed processing capacity; during traffic spikes (e.g., promotional events), the query rate can exceed this capacity, causing queuing delays, timeouts, and intermittent failures exactly as described.
- Cascading user-facing delays: Since DNS resolution is the first step in nearly every web request, a slow or overloaded DNS server delays every subsequent connection, amplifying the perceived slowness of the entire application even if the application servers themselves are healthy.

2. Proposed Distributed DNS Solution

- Deploy Anycast DNS across multiple geographically distributed data centers so the same DNS service IP is announced from many locations, and each user's query is automatically routed to the nearest healthy node by BGP.
- Use a multi-tier resolution architecture: local caching resolvers close to major user population centers, backed by regional authoritative servers, backed by a globally replicated master DNS dataset — so most queries are answered without crossing regions.
- Implement health-checked load balancing at the DNS layer (e.g., DNS-based load balancing/GSLB) so that if one regional node becomes overloaded or fails, traffic is automatically redirected to the next-nearest available node.
- Replicate DNS zone data across all nodes with fast, secure zone transfer mechanisms, ensuring every node can answer authoritative queries independently without relying on a single master server for every request.

3. Influence of TTL, Anycast Routing, and DNS Caching on Performance and Fault Tolerance

- TTL values: Shorter TTLs allow the system to redirect users away from a failing node more quickly (important for fault tolerance during an outage), but increase the query load on authoritative servers since cached answers expire sooner; longer TTLs reduce load and improve average performance but slow down failover response — an optimal balance (e.g., short TTLs specifically on load-balancing/failover-relevant records) is needed.
- Anycast routing: By directing each query to the nearest operational node, Anycast both reduces latency (performance) and automatically routes around a failed or unreachable node (fault tolerance) without requiring any client-side reconfiguration, since BGP simply stops advertising the failed node's route.
- DNS caching: Caching at resolvers and clients means that once a domain is resolved, subsequent requests are served instantly from local memory rather than generating new

network queries, which both reduces load on the DNS infrastructure and improves perceived performance — but stale cached entries can delay users from seeing an update if a node fails between refreshes, reinforcing the need for carefully tuned TTLs.

- Combined effect: Together, Anycast (routes around failures and reduces distance-based latency), sensible TTLs (balances freshness against load), and caching (reduces repeated query overhead) form a layered defense that keeps lookup times low under normal conditions and maintains availability during node failures or traffic surges.

Case Study 4: Increased Order Latency on a Stock Trading Platform (10 Marks)

A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the average order acknowledgment latency increases from 1 ms to 40 ms, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

1. Three Possible TCP-Related Factors Responsible for the Latency Increase

- Flow control: TCP's receive-window mechanism limits how much unacknowledged data the sender can transmit at once. Under the burst of order traffic at market open, if the receiver's advertised window becomes small (e.g., due to a slow-draining application buffer on the receiving trading server), the sender is forced to pause and wait, adding delay to every subsequent order even though the network itself may have spare capacity.
- Nagle's algorithm: Nagle's algorithm delays sending small TCP segments so they can be coalesced into fewer, larger packets, in order to reduce overhead for bulk transfers. For a trading platform sending many small, latency-sensitive order/acknowledgment messages, this delay (waiting to batch data or for an ACK of previous small data) directly adds tens of milliseconds of unnecessary latency to each transaction.
- SYN queue limitations: Each new TCP connection requires a three-way handshake, and incoming SYN requests are placed in a fixed-size SYN queue (backlog) at the server. During the surge of connections at market opening, if the SYN queue fills up, additional connection attempts are delayed or dropped and must be retried, significantly lengthening connection establishment time exactly as observed.

2. How Each Factor Affects Transaction Processing in High-Frequency Systems

- Flow control: A shrinking receive window under load throttles the sender even when there is no actual network congestion, meaning legitimate, time-critical orders are queued and delayed purely due to buffering/window constraints on the receiving side, directly increasing the observed acknowledgment latency.

- Nagle's algorithm: Because it is designed to optimize for throughput rather than latency, Nagle's algorithm is fundamentally at odds with high-frequency trading, where each order/acknowledgment is small and time-critical; the artificial delay it introduces (which can be tens of milliseconds when combined with delayed ACKs) is unacceptable when even 1 ms of added delay affects trade outcomes.
- SYN queue limitations: In systems where new connections are frequently established (e.g., short-lived sessions or reconnect storms during high load), a saturated SYN queue causes connection setup itself — which should take a fraction of a millisecond on a fast network — to balloon into tens of milliseconds of extra delay, directly delaying the first order sent on that connection and any dependent confirmations.

3. Recommended TCP Configuration Changes

- Disable Nagle's algorithm (set TCP_NODELAY) on the trading platform's sockets so small order and acknowledgment messages are sent immediately rather than being buffered and coalesced, directly reducing per-message latency.
- Also disable or tune delayed ACKs (TCP_QUICKACK where available) in conjunction with TCP_NODELAY, since the interaction between Nagle's algorithm and delayed ACKs is often the actual source of tens-of-milliseconds stalls.
- Increase the TCP receive buffer / window size on the receiving trading servers (and enable window scaling) so that flow control does not unnecessarily throttle the sender during traffic bursts at market open.
- Increase the SYN backlog queue size (e.g., via the operating system's connection backlog and SOMAXCONN settings) and enable SYN cookies to handle a burst of new connections at market opening without dropping or delaying handshake requests.
- Use persistent, pre-established TCP connections (connection pooling/keep-alive) between trading clients and servers wherever possible, so that the cost of connection establishment (including any SYN queue delay) is paid once rather than repeatedly during the high-volume opening period.
- Tune retransmission timeout (RTO) parameters and enable modern congestion control algorithms (e.g., BBR) suited to low-latency, high-throughput environments, so that any genuine packet loss is detected and recovered from more quickly than with default settings.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand